

Password Cracking su DVWA

Studente: Simone Santonico

Data:14/05/2026

1. Scopo del laboratorio

L'obiettivo dell'attività è simulare un'operazione di password auditing su un ambiente vulnerabile (DVWA).

Lo studente deve:

- accedere al database dell'applicazione
- estrarre gli hash delle password
- identificarne il formato
- eseguire un attacco di cracking
- analizzare i risultati e le implicazioni di sicurezza

Questo esercizio riproduce una fase reale di un penetration test: la compromissione delle credenziali tramite attacco offline.

2. Accesso al database e raccolta degli hash

Dopo l'avvio del servizio MariaDB, l'accesso è stato effettuato tramite l'utente root.

Una volta selezionato il database DVWA, è stata individuata la tabella "users", contenente gli hash delle password.

Il comando utilizzato per estrarre i dati è stato:

```
SELECT user, password FROM users;
```

Gli hash recuperati erano tutti in formato MD5, riconoscibile dalla lunghezza fissa di 32 caratteri esadecimali.

Questa informazione è fondamentale per selezionare il corretto modulo di cracking.

3. Preparazione del file per il cracking

Gli hash sono stati copiati in un file di testo denominato `dvwa_hashes.txt`, uno per riga.

Questa procedura permette di eseguire un attacco offline, evitando qualsiasi interazione con il sistema target e riducendo il rischio di rilevamento.

4. Attacco di cracking con John the Ripper

Per avviare il cracking è stato utilizzato il comando:

```
john --format=raw-md5 dvwa_hashes.txt
```

John the Ripper ha caricato gli hash e ha avviato un attacco basato su dizionario.

Grazie alla debolezza dell'algoritmo MD5 e alla scarsa complessità delle password, il cracking è stato completato in pochi istanti.

I risultati sono stati visualizzati tramite:

```
john --show dvwa_hashes.txt
```

5. Password recuperate

Tutte le password sono state crackate con successo.

Gli utenti utilizzavano password estremamente deboli, tra cui:

- "password"
- "abc123"
- "1337"
- "letmein"

Queste password sono presenti in qualsiasi dizionario di attacco e vengono recuperate immediatamente.

6. Analisi delle vulnerabilità

L'esercizio evidenzia diverse criticità:

1. **Uso di MD5**
MD5 è un algoritmo obsoleto, vulnerabile a collisioni e attacchi di forza bruta.
Non deve essere utilizzato per proteggere password.
2. **Password deboli e prevedibili**
Password semplici vengono crackate in pochi millisecondi.
In un contesto reale, ciò permetterebbe a un attaccante di ottenere accesso completo al sistema.
3. **Assenza di salting**
Gli hash non sono salati, quindi identiche password generano hash identici.
Questo facilita attacchi con rainbow tables.
4. **Mancanza di policy di sicurezza**
L'applicazione non impone complessità minima né controlli di robustezza.

7. Best practice raccomandate

Per mitigare le vulnerabilità emerse, si raccomanda:

- utilizzo di algoritmi moderni come **bcrypt**, **scrypt**, **PBKDF2** o **Argon2**
- implementazione di password policy robuste
- obbligo di password uniche e non riutilizzate
- salting e peppering degli hash
- monitoraggio degli accessi e dei tentativi di login
- formazione degli utenti sulla sicurezza delle credenziali

8. Conclusioni

Il laboratorio ha dimostrato in modo pratico come un attaccante possa compromettere rapidamente un sistema che utilizza hashing debole e password semplici.

L'esercizio riproduce una fase reale di un penetration test e sottolinea l'importanza di adottare tecniche moderne di protezione delle credenziali.

L'attività è stata completata con successo: gli hash sono stati estratti, analizzati, crackati e valutati dal punto di vista della sicurezza.

```
kali@kali: ~  
└─$ sudo apt install john -y  
Error: Unable to locate package john  
  
kali@kali:~$  
└─$ sudo apt install john -y  
john is already the newest version (1.9.0-Jumbo-1-git20211102-0kali10).  
john set to manually installed.  
Summary:  
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1306  
  
kali@kali:~$  
└─$ john --show dowa_hashes.txt  
Created directory: /home/kali/.john  
0 password hashes cracked, 10 left  
  
kali@kali:~$  
└─$ john --format=raw-md5 dowa_hashes.txt  
Using default input encoding: UTF-8  
Loaded 5 password hashes with no different salts (Raw-MD5 [MD5 256/256 AVX2 8x3])  
Warning: no OpenMP support for this hash type, consider --fork=2  
Proceeding with single, rules:Single  
Press 'q' or ctrl-C to abort, almost any other key for status  
Almost done: Processing the remaining buffered candidate passwords, if any.  
Proceeding with wordlist:/usr/share/john/password.lst  
password      (?)  
password      (?)  
abc123        (?)  
letmein       (?)  
Proceeding with incremental:ASCII  
charley       (?)  
bg 0:00:00:00 DONE 3/3 (2026-05-14 00:00) 20.00g/s 713400p/s 713400c/s 719544C/s stevy13..candake  
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably  
Session completed.  
  
kali@kali:~$  
└─$ john --show dowa_hashes.txt  
0 password hashes cracked, 10 left  
  
kali@kali:~$  
└─$ john --format=raw-md5 dowa_hashes.txt  
Using default input encoding: UTF-8  
Loaded 5 password hashes with no different salts (Raw-MD5 [MD5 256/256 AVX2 8x3])  
No password hashes left to crack (see FAQ)  
  
kali@kali:~$  
└─$ john --show dowa_hashes.txt  
0 password hashes cracked, 10 left  
  
kali@kali:~$  
└─$
```